

UniFACEF

Engenharia de Software

RELATÓRIO DA UCE
PARADIGMAS DE PROGRAMAÇÃO I
IMPLEMENTAÇÃO DE API REST COM ARQUITETURA EM
CAMADAS

LUCAS ANTONIO MONTEIRO
MATHEUS LOMBARDI DE SOUZA
RAFAEL MENDES FERREIRA
GUILHERME DE OLIVEIRA PARREIRA

SUMÁRIO

- 1 Introdução**
- 2 Arquitetura em Camadas: Por Que Usar?**
 - 2.1 O que é arquitetura em camadas?
 - 2.2 Benefícios práticos
 - 2.3 O que aconteceria sem essa organização?
- 3 Diagrama de Classes e Relacionamentos**
- 4 Camada Model**
- 5 Camada Repository**
- 6 Camada Service**
- 7 Camada Controller**
- 8 Fluxo Completo de uma Requisição**
- 9 Testes Realizados**
- 10 Conclusão**
- Referências**

1. INTRODUÇÃO

O projeto desenvolvido tem como domínio a gestão de personal trainers e seus alunos, abrangendo tanto o atendimento presencial quanto o online. O sistema foi concebido para apoiar as atividades do Personal Trainer Tiago, profissional autônomo da área de educação física que oferece acompanhamento individualizado de treinamento físico.

As entidades principais do sistema são: Aluno, Treinos, Exercício, ExercícioTreino, Planos e AgendaTreino. Juntas, essas entidades compõem um modelo capaz de representar toda a dinâmica entre o profissional, seus alunos e os programas de treinamento.

A empresa escolhida é o Personal Trainer Tiago, um negócio de prestação de serviços na área de educação física e condicionamento físico. O profissional atua no segmento de atividade física personalizada, oferecendo acompanhamento tanto na modalidade presencial quanto online, adaptando os treinos às necessidades individuais de cada aluno. A atividade econômica enquadra-se na prestação de serviços de saúde, bem-estar e condicionamento físico.

O sistema, denominado AlphaCoach, é uma plataforma de gestão de alunos e treinos voltada para personal trainers. Seu propósito é centralizar em um único ambiente todas as informações referentes aos alunos — como dados cadastrais, planos contratados e agenda de treinos — além de permitir a montagem, organização e acompanhamento de treinos de forma estruturada e eficiente. O sistema também controla exercícios disponíveis, séries, repetições, cargas e o progresso de cada aluno ao longo do tempo.

Antes da implementação do sistema, o personal trainer enfrentava dificuldades na organização e agilidade da montagem de treinos, bem como na gestão descentralizada dos dados dos alunos, que frequentemente ficavam dispersos em planilhas, cadernos ou aplicativos distintos. O sistema proposto busca apoiar justamente esses dois pontos críticos: acelerar a elaboração de programas de treino e centralizar o controle dos alunos em um único lugar, proporcionando maior eficiência operacional e melhor experiência tanto para o profissional quanto para os alunos.

As classes modeladas no diagrama de classe do sistema são:

- **Aluno** — representa o cliente do personal trainer, contendo dados pessoais, plano contratado, objetivos e agenda;
- **Treinos** — representa um programa de treino vinculado a um aluno, composto por exercícios e com controle de status e data de criação;

- **ExercicioTreino** — classe associativa que detalha a execução de um exercício dentro de um treino, com informações de séries, repetições, carga, intensidade e potência;
- **Exercicio** — representa o catálogo de exercícios disponíveis, com nome, descrição, link de vídeo e status;
- **Planos** — representa os planos de assinatura disponíveis, com valor, duração, tipo e desconto aplicável;
- **AgendaTreino** — representa o agendamento de sessões de treino do aluno, com controle de presença, faltas e remarcações.

O banco de dados utilizado foi o PostgreSQL, um sistema gerenciador de banco de dados relacional de código aberto, reconhecido por sua robustez e conformidade com o padrão SQL. A configuração foi realizada por meio do framework Spring Boot, utilizando as propriedades do arquivo `application.properties` para definir a URL de conexão, credenciais de acesso, dialeto do Hibernate e estratégia de criação/atualização do esquema. O mapeamento objeto-relacional foi gerenciado pelo JPA (Java Persistence API) com a implementação do Hibernate, que foi responsável por gerar e sincronizar as tabelas automaticamente a partir das entidades Java.

As tabelas presentes no banco de dados do sistema são:

- **alunos** — armazena os dados cadastrais dos alunos;
- **treinos** — armazena os treinos criados para cada aluno;
- **exercicios** — armazena o catálogo de exercícios disponíveis;
- **exercicio_treino** — tabela associativa que relaciona exercícios a treinos com seus parâmetros de execução;
- **planos** — armazena os planos de assinatura disponíveis;
- **agenda_aluno** — armazena os agendamentos de treino dos alunos.

Os relacionamentos entre as tabelas seguem a estrutura definida no diagrama de classes:

- **Planos** → **Alunos**: um plano pode ser associado a múltiplos alunos (1:N);
- **Alunos** → **Treinos**: um aluno pode possuir múltiplos treinos (1:N);
- **Treinos** → **ExercicioTreino**: um treino é composto por múltiplos registros de exercícios (1:N), com composição forte;
- **ExercicioTreino** → **Exercicios**: cada registro de exercício no treino referencia um exercício do catálogo (N:1);
- **Alunos** → **AgendaTreino**: um aluno pode ter múltiplos agendamentos de treino (1:N), com composição forte.

As tecnologias e frameworks empregados no desenvolvimento do sistema foram:

- **Java 21** — linguagem de programação principal do back-end;
- **Spring Boot** — framework para criação e configuração da aplicação de forma ágil;
- **JPA (Java Persistence API)** — especificação para mapeamento objeto-relacional e persistência de dados;
- **Hibernate** — implementação do JPA, responsável pela geração e manipulação das tabelas no banco de dados;
- **PostgreSQL** — banco de dados relacional utilizado para armazenamento das informações.

O código-fonte do projeto está disponível publicamente no GitHub, podendo ser acessado pelo seguinte endereço:
<https://github.com/Lucas-Monte/AlphaCoach-TIC>

2. ARQUITETURA EM CAMADAS: POR QUE USAR?

2.1 O que é arquitetura em camadas?

A arquitetura em camadas é um padrão de projeto de software baseado no princípio de Separação de Responsabilidades (*Separation of Concerns*), que estabelece que cada parte do sistema deve ter uma responsabilidade única e bem definida, sem interferir nas responsabilidades das demais. Esse princípio torna o código mais organizado, legível, testável e fácil de manter, pois alterações em uma camada não propagam impactos desnecessários às outras.

No projeto AlphaCoach, a arquitetura foi estruturada em quatro camadas, cada uma com um papel específico dentro do fluxo da aplicação:

Camada	Responsabilidade Principal	Anotações Utilizadas
Model	Representa as entidades do domínio do sistema, mapeando os objetos Java às tabelas do banco de dados. É nessa camada que são definidos os atributos, relacionamentos e regras estruturais de cada entidade, como Aluno, Treinos, Exercicios, ExercicioTreino, Planos e AgendaTreino.	@Entity, @Table, @Id, @GeneratedValue, @ManyToOne, @OneToMany, @Column
Repository	Responsável pela comunicação direta com o banco de dados. Contém as interfaces que herdam de JpaRepository, fornecendo operações de persistência prontas (como salvar, buscar, atualizar e deletar) sem necessidade de escrever SQL manualmente. Exemplos: AlunoRepository, TreinosRepository.	@Repository
Service	Concentra as regras de negócio da aplicação. É a camada intermediária entre o Controller e o Repository, onde ficam as lógicas de validação, cálculos e orquestração das operações. Exemplos: AlunoService, ExercicioTreinoService.	@Service, @Transactional
Controller	Responsável por receber as requisições HTTP enviadas pelo cliente, delegar o processamento à camada de serviço e retornar as respostas adequadas. Define os endpoints da API REST. Exemplos: AlunoController, TreinosController.	@RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping

Essa separação garante que, por exemplo, uma mudança na regra de negócio afete apenas a camada **Service**, sem impactar o **Controller** ou o **Repository**, tornando o sistema mais robusto e preparado para evoluções futuras.

2.2 Benefícios práticos

A seguir são apresentados os principais benefícios percebidos ao organizar o código do projeto AlphaCoach em camadas, ilustrados com exemplos práticos do próprio sistema.

Manutenção

A separação em camadas permite alterar uma parte do sistema sem comprometer as demais. No projeto AlphaCoach, suponha que seja necessário modificar a regra de cálculo do valor mensal de um plano, por exemplo, aplicar um novo critério de desconto progressivo. Essa alteração ficaria restrita ao PlanosService, sem que o PlanosController ou o PlanosRepository precisem ser tocados. Da mesma forma, se o banco de dados precisasse ser migrado do PostgreSQL para outro SGBD, apenas a camada de Repository e as configurações de conexão seriam afetadas, preservando integralmente as regras de negócio e os endpoints da API.

Reaproveitamento

Um Service pode ser injetado e utilizado por mais de um Controller, evitando a duplicação de código. No AlphaCoach, o AlunoService concentra toda a lógica relacionada ao ciclo de vida de um aluno, atualização de status, mudança de plano. Caso futuramente seja criado um painel administrativo com seu próprio Controller, ele poderia reutilizar o mesmo AlunoService já existente, sem reescrever nenhuma regra de negócio. O mesmo vale para o ExercicioTreinoService, que pode ser acionado tanto pelo ExercicioTreinoController quanto por um eventual módulo de relatórios de desempenho do aluno.

Testabilidade

A organização em camadas facilita a escrita de testes ao permitir que cada camada seja testada de forma isolada. No projeto AlphaCoach, é possível testar o TreinosService verificando se a lógica de finalização de um treino ou o cálculo do volume total está correta, sem precisar subir o servidor ou conectar ao banco de dados. Da mesma forma, o TreinosController pode ser testado verificando apenas se os endpoints respondem corretamente aos códigos HTTP esperados, sem depender da implementação real do Service. Essa independência entre camadas torna os testes mais rápidos, simples e confiáveis.

Legibilidade

A estrutura em camadas funciona como uma convenção de organização que qualquer desenvolvedor familiarizado com Spring Boot reconhece imediatamente. Ao abrir o projeto AlphaCoach, um novo desenvolvedor consegue identificar rapidamente onde encontrar cada tipo de código: os endpoints estão nos Controllers, as regras de negócio nos Services, o acesso ao banco nos Repositories e as entidades nos Models. Por exemplo, se há um bug na resposta da API de agendamento, o desenvolvedor sabe que deve procurar no AgendaTreinoController; se é um problema de lógica, vai ao AgendaTreinoService; se é de persistência, ao AgendaTreinoRepository. Isso reduz drasticamente o tempo de onboarding e de diagnóstico de problemas.

Escalabilidade

A arquitetura em camadas prepara o sistema para crescer sem perder organização. No AlphaCoach, caso o personal trainer expanda seu negócio e precise de novos módulos basta criar novos Models, Repositories, Services e Controllers correspondentes, sem modificar o que já funciona. A camada de StatusPagamento, já prevista no diagrama de classes, é um exemplo claro dessa escalabilidade: ela pode ser desenvolvida e conectada ao sistema futuramente de forma independente, aproveitando a estrutura já consolidada das demais entidades.

2.3 O que aconteceria sem essa organização?

Para ilustrar os riscos dessa abordagem, imagine que toda a lógica do AlphaCoach estivesse concentrada em uma única classe responsável ao mesmo tempo por receber requisições HTTP, aplicar regras de negócio e executar queries no banco de dados.

Com tudo misturado em um único lugar, qualquer alteração simples se tornaria arriscada. Modificar a regra de cálculo do valor mensal de um plano exigiria mexer no mesmo bloco de código que trata requisições HTTP e executa comandos SQL, de modo que uma mudança pontual poderia acidentalmente quebrar um endpoint completamente não relacionado, como o cadastro de alunos. O que hoje é uma alteração segura e isolada no PlanosService se tornaria uma operação de alto risco.

Sem uma camada de Service centralizada, a mesma lógica precisaria ser repetida em vários pontos do sistema. A regra de verificar se um aluno está ativo antes de montar um treino teria que ser reescrita em cada trecho do código que realizasse essa operação, e com o tempo pequenas variações entre essas cópias gerariam comportamentos inconsistentes.

Um novo desenvolvedor que entrasse no projeto encontraria um único arquivo com centenas ou milhares de linhas misturando SQL, regras de negócio e configurações de rotas HTTP. No AlphaCoach, que já possui seis entidades principais, cada uma com suas próprias regras, relacionamentos e endpoints, essa estrutura rapidamente se tornaria incompreensível. À medida que o negócio crescesse, adicionar funcionalidades nesse cenário significaria aumentar ainda mais a complexidade de um código já desorganizado, tornando cada nova entrega progressivamente mais lenta e arriscada.

A ausência de uma arquitetura em camadas, portanto, não representa apenas um problema de organização interna do código, mas um risco concreto à qualidade, à confiabilidade e à longevidade do sistema. O que aparenta ser uma simplificação no início do desenvolvimento tende a se transformar, com o crescimento do projeto, em um obstáculo que compromete cada manutenção, cada nova funcionalidade e cada correção de falha identificada.

na tabela `exercicio_treino` referencia o id da tabela `treinos`, garantindo que a exclusão de um treino implique também a remoção de todos os seus exercícios associados.

ExercicioTreino → Exercicio (N:1)

Múltiplos registros de `ExercicioTreino` podem referenciar o mesmo exercício do catálogo, mas cada `ExercicioTreino` está vinculado a apenas um exercício. Essa cardinalidade permite que um mesmo exercício — como agachamento ou supino — seja reutilizado em diferentes treinos e para diferentes alunos, sem necessidade de cadastrá-lo. A chave estrangeira `exercicio_id` na tabela `exercicio_treino` referencia o id da tabela `exercicios`.

Aluno → AgendaTreino (1:N, Composição)

Um aluno pode possuir múltiplos agendamentos de treino, enquanto cada agendamento pertence a um único aluno. O relacionamento foi modelado como composição, pois um agendamento não tem razão de existir sem o aluno associado. A exclusão de um aluno implica, portanto, a remoção de toda a sua agenda. A chave estrangeira `aluno_id` na tabela `agenda_aluno` referencia o id da tabela `alunos`.

4. CAMADA MODEL

A camada Model representa as entidades do domínio e é responsável por mapear objetos Java para as tabelas do banco de dados via JPA/Hibernate.

4.1 Alunos

Item	Descrição
Tabela no banco	alunos
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	@Entity, @Table(name="alunos ")

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	@Id, @GeneratedValue(IDENTITY)	Chave primária gerada automaticamente
nome	String	@Column(nullable=false)	Nome obrigatório
email	String	@Column(nullable=false)	E-mail do aluno
cpf	String	@Column	CPF do aluno, único no sistema
dataNascimento	LocalDate	@Column	Data de nascimento do aluno
endereço	String	@Column	Endereço do aluno
tipoCliente	String	@Column	Define se o atendimento é presencial ou online
ativo	boolean	@Column	Indica se o aluno está ativo no sistema
telefone	String	@Column	Telefone de contato do aluno
plano	Planos	@ManyToOne, @JoinColumn	Plano contratado pelo aluno
objetivo	String	@Column	Objetivo do aluno (ex: hipertrofia, emagrecimento)
anamnese	String	@Column	Histórico de saúde e informações clínicas do aluno
agenda	List<Agenda Treino>	@OneToMany(mappedBy="aluno")	Lista de agendamentos do aluno

Relacionamentos mapeados nesta classe:

A classe Aluno possui dois relacionamentos mapeados. O primeiro é com a classe Planos, utilizando a anotação @ManyToOne acompanhada de @JoinColumn(name="plano_id"), indicando que muitos alunos podem estar associados a um mesmo plano. O @JoinColumn define que a chave estrangeira plano_id ficará armazenada na própria tabela alunos, sendo o lado proprietário do relacionamento. O

segundo relacionamento é com AgendaTreino, mapeado com `@OneToMany(mappedBy="aluno")`, indicando que um aluno pode ter múltiplos agendamentos. O atributo `mappedBy="aluno"` informa ao JPA que o lado proprietário do relacionamento está na classe AgendaTreino, evitando a criação de uma tabela intermediária desnecessária. Em ambos os casos, `@JsonIgnoreProperties` pode ser utilizado para evitar referências circulares na serialização JSON.

4.2 Planos

Item	Descrição
Tabela no banco	planos
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	<code>@Entity, @Table(name="planos")</code>

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	<code>@Id, @GeneratedValue(IDENTITY)</code>	Chave primária gerada automaticamente
descricao	String	<code>@Column(nullable=false)</code>	Descrição do plano
valor	float	<code>@Column(nullable=false)</code>	Valor mensal do plano
duracaoMeses	int	<code>@Column</code>	Duração do plano em meses
tipo	String	<code>@Column</code>	Tipo do plano (ex: mensal, trimestral)
ativo	boolean	<code>@Column</code>	Indica se o plano está disponível para contratação

Relacionamentos mapeados nesta classe:

A classe Planos não possui relacionamentos mapeados diretamente em seus atributos. O relacionamento com Aluno é gerenciado pelo lado proprietário, que está na classe Aluno por meio da chave estrangeira `plano_id`. Dessa forma, Planos atua como o lado inverso do relacionamento, sem necessidade de declarar uma lista de alunos em sua estrutura.

4.3 Treinos

Item	Descrição
Tabela no banco	treinos
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	<code>@Entity, @Table(name="treinos")</code>

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	@Id, @GeneratedValue(IDENTITY)	Chave primária gerada automaticamente
nome	String	@Column(nullable=false)	Nome do treino (ex: Treino A - Peito)
exercicios	List<ExercicioTreino>	@OneToMany(mappedBy="treino")	Lista de exercícios do treino
aluno	Aluno	@ManyToOne, @JoinColumn	Aluno ao qual o treino pertence
dataCriacao	LocalDate	@Column	Data de criação do treino
status	boolean	@Column	Indica se o treino está ativo ou finalizado

Relacionamentos mapeados nesta classe:

A classe Treinos possui dois relacionamentos. O primeiro é com Aluno, mapeado com @ManyToOne e @JoinColumn(name="aluno_id"), definindo que muitos treinos podem pertencer a um mesmo aluno e que a chave estrangeira aluno_id ficará na tabela treinos. O segundo é com ExercicioTreino, mapeado com @OneToMany(mappedBy="treino") e com cascade = CascadeType.ALL, indicando que um treino pode conter múltiplos exercícios e que operações como exclusão se propagam automaticamente para os registros filhos. O mappedBy="treino" indica que o lado proprietário está em ExercicioTreino.

4.4 ExercicioTreino.

Item	Descrição
Tabela no banco	exercicio_treino
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	@Entity, @Table(name="exercicio_treino")

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	@Id, @GeneratedValue(IDENTITY)	Chave primária gerada automaticamente
exercicio	Exercicios	@ManyToOne, @JoinColumn	Exercício referenciado do catálogo
treino	Treinos	@ManyToOne, @JoinColumn	Treino ao qual este registro pertence
potencia	float	@Column	Potência aplicada no exercício
intensidade	float	@Column	Intensidade do exercício
series	int	@Column	Número de séries

repeticoes	int	@Column	Número de repetições por série
carga	String	@Column	Carga utilizada no exercício
tempoDescanso	int	@Column	Tempo de descanso em segundos entre séries
status	boolean	@Column	Indica se o exercício foi concluído no treino

Relacionamentos mapeados nesta classe:

A classe ExercicioTreino é o lado proprietário de dois relacionamentos. O primeiro é com Treinos, mapeado com @ManyToOne e @JoinColumn(name="treino_id"), armazenando a chave estrangeira que vincula o registro ao seu treino pai. O segundo é com Exercicios, mapeado com @ManyToOne e @JoinColumn(name="exercicio_id"), referenciando o exercício do catálogo. Em ambos os casos, @JsonIgnoreProperties é utilizado para evitar loops infinitos durante a serialização, uma vez que Treinos também referencia ExercicioTreino em sua lista de exercícios.

4.5 Exercicios

Item	Descrição
Tabela no banco	exercicios
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	@Entity, @Table(name="exercicios")

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	@Id, @GeneratedValue(IDENTITY)	Chave primária gerada automaticamente
nome	String	@ManyToOne, @JoinColumn	Nome do exercício
descricao	String	@Column	Descrição do exercício
ativo	boolean	@Column	Indica se o exercício está disponível para uso
linkVideo	String	@Column	Link de vídeo demonstrativo do exercício

Relacionamentos mapeados nesta classe:

A classe Exercicios não possui relacionamentos mapeados diretamente em seus atributos. Ela atua como entidade de catálogo, sendo referenciada por ExercicioTreino por meio da chave estrangeira exercicio_id. Por ser o lado inverso do

relacionamento, não há necessidade de declarar uma coleção de `ExercicioTreino` em sua estrutura, mantendo a classe simples e coesa.

4.6 AgendaTreino

Item	Descrição
Tabela no banco	agenda_aluno
Herda de	nenhuma
Subclasses	nenhuma
Anotação principal	@Entity, @Table(name="agenda_aluno")

Atributos e suas anotações JPA:

Atributo	Tipo	Anotações JPA	Descrição
id	Long	@Id, @GeneratedValue(IDENTITY)	Chave primária gerada automaticamente
aluno	Aluno	@ManyToOne, @JoinColumn	Aluno ao qual o agendamento pertence
data	LocalDateTime	@Column	Data e hora do agendamento
checkIn	boolean	@Column	Indica se o aluno confirmou presença na sessão

Relacionamentos mapeados nesta classe:

A classe `AgendaTreino` possui um relacionamento com `Aluno`, mapeado com `@ManyToOne` e `@JoinColumn(name="aluno_id")`, definindo que muitos agendamentos podem pertencer a um mesmo aluno. A chave estrangeira `aluno_id` fica armazenada na tabela `agenda_aluno`, sendo esta a classe proprietária do relacionamento. `@JsonIgnoreProperties` é aplicado para evitar referências circulares na serialização, dado que `Aluno` também referencia `AgendaTreino` em sua lista de agendamentos.

5. CAMADA REPOSITORY

A camada Repository é responsável pelo acesso ao banco de dados. Com Spring Data JPA, operações CRUD básicas são geradas automaticamente, e consultas personalizadas podem ser criadas de forma simples.

5.1 AlunoRepository

Interface pai e o que ela fornece:

A interface `AlunoRepository` estende `JpaRepository<Aluno, Long>`, fornecendo automaticamente todas as operações CRUD para a tabela alunos. Além dos métodos herdados, esta interface declara o método personalizado `existsByCpf(String cpf)`, que verifica se já existe um aluno cadastrado com determinado CPF. Esse método utiliza a convenção de nomenclatura do Spring Data JPA, que interpreta o nome do método e gera automaticamente a query equivalente, neste caso, um `SELECT EXISTS` filtrando pelo campo `cpf`. Esse método é especialmente importante para garantir a unicidade do CPF no momento do cadastro de um novo aluno, evitando duplicidades no sistema.

5.2 TreinosRepository

Interface pai e o que ela fornece:

A interface `TreinosRepository` estende `JpaRepository<Treinos, Long>`, fornecendo automaticamente as operações de persistência para a tabela treinos. Por meio dessa interface, o sistema consegue salvar novos treinos, buscar treinos por id, listar todos os treinos cadastrados e remover treinos existentes sem nenhum código adicional.

5.3 ExerciciosRepository

Interface pai e o que ela fornece:

A interface `ExerciciosRepository` estende `JpaRepository<Exercicios, Long>`, gerenciando as operações de persistência sobre a tabela exercicios. Além dos métodos CRUD padrão, esta interface declara o método personalizado `existsByNome(String nome)`, que verifica se já existe um exercício cadastrado com determinado nome. Assim como no `AlunoRepository`, o Spring Data JPA interpreta a nomenclatura do método e gera a query automaticamente, garantindo que o catálogo de exercícios não contenha registros duplicados.

5.4 ExercicioTreinoRepository

Interface pai e o que ela fornece:

A interface `ExercicioTreinoRepository` estende `JpaRepository<ExercicioTreino, Long>`, gerenciando os registros da tabela `exercicio_treino`. Essa interface é fundamental para o funcionamento do sistema, pois é por meio dela que os exercícios são vinculados aos treinos, com seus respectivos parâmetros de séries, repetições, carga e tempo de descanso.

5.5 PlanosRepository

Interface pai e o que ela fornece:

A interface `PlanosRepository` estende `JpaRepository<Planos, Long>`, gerenciando as operações sobre a tabela `planos`. Por meio dos métodos herdados, o sistema consegue cadastrar novos planos, listá-los para exibição e removê-los quando necessário.

5.6 AgendaTreinoRepository

Interface pai e o que ela fornece:

A interface `AgendaTreinoRepository` estende `JpaRepository<AgendaTreino, Long>`, gerenciando os registros da tabela `agenda_aluno`. Além dos métodos CRUD padrão, esta interface declara o método personalizado `existsByAlunoAndData(Aluno aluno, LocalDateTime data)`, que verifica se já existe um agendamento para um determinado aluno em uma data e horário específicos. Esse método é fundamental para evitar conflitos de agenda, impedindo que o mesmo aluno seja agendado duas vezes para o mesmo horário. O Spring Data JPA combina os dois parâmetros, `aluno` e `data`, gerando automaticamente uma query com dupla condição de filtragem.

6. CAMADA SERVICE

A camada Service contém a lógica de negócio da aplicação. Ela intermedia a comunicação entre o Controller e o Repository. É aqui que regras de negócio, validações e controle de transações devem ser implementados.

6.1 AlunoService

Anotações da classe:

A classe AlunoService é registrada no IoC Container do Spring pela anotação `@Service`, permitindo que seja injetada no AlunoController. Ela depende do AlunoRepository para acessar o banco de dados, injetado via construtor.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca por id, retorna <code>Optional<Aluno></code>	—
buscarPorId(Long id)	Sim	Persiste o objeto no banco	Verifica se já existe um aluno com o mesmo CPF via <code>existsByCpf()</code> antes de salvar
remover(Long id)	Sim	Remove o registro pelo id	Verifica se o aluno existe antes de remover
alterar(Long id, Aluno aluno)	Sim	Atualiza campos do registro existente	Verifica existência antes de atualizar

Por que usar @Transactional?

Uma transação de banco de dados é uma unidade de trabalho que deve ser executada de forma completa ou não executada de forma alguma. O padrão ACID (Atomicidade, Consistência, Isolamento e Durabilidade) garante que operações sobre o banco de dados sejam confiáveis e seguras. A anotação `@Transactional` instrui o Spring a abrir uma transação antes da execução do método e confirmá-la ao final (commit) caso tudo ocorra sem erros, ou desfazê-la (rollback) em caso de exceção. No AlunoService, sem essa anotação, uma falha durante o processo de salvar um aluno poderia resultar em um registro incompleto no banco, comprometendo a integridade dos dados.

6.2 AgendaTreinoService

Anotações da classe:

A classe AgendaTreinoService é registrada no IoC Container do Spring pela anotação `@Service`, sendo injetada no AgendaTreinoController. Ela depende do AgendaTreinoRepository para gerenciar os agendamentos de treino.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca um agendamento por id, retorna Optional<AgendaTreino>	—
salvar(AgendaTreino agenda)	Sim	salvar(AgendaTreino agenda)	Verifica se já existe agendamento para o mesmo aluno no mesmo horário via existsByAlunoAndData()
remover(Long id)	Sim	Remove o registro pelo id	Verifica existência antes de remover
alterar(Long id, AgendaTreino agenda)	Sim	Atualiza campos do registro existente	Verifica existência antes de atualizar

Por que usar @Transactional?

No AgendaTreinoService, a anotação `@Transactional` é especialmente importante no método `salvar()`, pois ele envolve múltiplas verificações, como checar a existência do aluno e a disponibilidade do horário via `existsByAlunoAndData()`, antes de persistir o agendamento. Sem essa anotação, uma falha em qualquer etapa intermediária poderia deixar o banco em estado inconsistente, com registros parcialmente criados ou verificações de duplicidade ignoradas.

6.3 ExerciciosService

Anotações da classe:

A classe ExerciciosService é registrada no IoC Container do Spring pela anotação `@Service`, sendo injetada no ExerciciosController. Ela depende do ExerciciosRepository para gerenciar o catálogo de exercícios.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca um exercício por id, retorna Optional<Exercicios>	—
salvar(Exercicios exercicio)	Sim	Persiste o objeto no banco	Verifica se já existe um exercício com o mesmo nome via existsByNome() antes de salvar
remover(Long id)	Sim	Remove o registro pelo id	Verifica se o exercício existe antes de atualizar
alterar(Long id, Exercicios exercicio)	Sim	Atualiza campos do registro existente	Verifica existência antes de atualizar

Por que usar @Transactional?

No ExerciciosService, a anotação @Transactional garante que o método salvar(), que verifica a existência de um exercício com o mesmo nome antes de persistir, seja executado de forma atômica. Sem ela, uma condição de corrida poderia permitir que dois exercícios com o mesmo nome fossem inseridos simultaneamente, violando a regra de unicidade do catálogo.

6.4 ExercicioTreinoService**Anotações da classe:**

A classe ExercicioTreinoService é registrada no IoC Container do Spring pela anotação @Service, sendo injetada no ExercicioTreinoController. Ela depende do ExercicioTreinoRepository para realizar suas operações.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca um registro por id, retorna Optional<ExercicioTreino>	—
salvar(ExercicioTreino et)	Sim	Associa um exercício a um treino com seus parâmetros	Verifica se o treino e o exercício existem antes de salvar
remover(Long id)	Sim	Remove o registro pelo id	Verifica se o registro existe antes de remover
alterar(Long id, ExercicioTreino et)	Sim	Atualiza parâmetros do exercício no treino	Verifica existência antes de atualizar

Por que usar @Transactional?

No ExercicioTreinoService, a anotação @Transactional é fundamental pois o método salvar() envolve a consulta de múltiplas entidades, treino e exercício, antes de persistir o vínculo entre elas. Sem essa anotação, uma falha após a verificação mas antes da persistência poderia resultar em dados inconsistentes, com vínculos parcialmente criados entre exercícios e treinos.

6.5 PlanosService

Anotações da classe:

A classe PlanosService é registrada no IoC Container do Spring pela anotação @Service, sendo injetada no PlanosController. Ela depende do PlanosRepository para gerenciar os planos de assinatura disponíveis.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca um plano por id, retorna Optional<Planos>	—
salvar(Planos plano)	Sim	Persiste o objeto no banco	Valida campos obrigatórios como valor e duração antes de salvar
remover(Long id)	Sim	Remove o registro pelo id	Verifica se o plano existe antes de atualizar
alterar(Long id, Planos plano)	Sim	Atualiza campos do registro existente	Verifica existência antes de atualizar

Por que usar @Transactional?

No PlanosService, a anotação @Transactional assegura que operações de escrita como salvar() e alterar() sejam executadas de forma atômica. Sem ela, em um cenário de múltiplos acessos simultâneos, atualizações concorrentes sobre o mesmo plano poderiam gerar resultados incorretos, comprometendo a integridade das informações financeiras do sistema.

6.6 TreinosService

Anotações da classe:

A classe TreinosService é registrada no IoC Container do Spring pela anotação @Service, sendo injetada no TreinosController. Ela depende do TreinosRepository para realizar suas operações.

Métodos implementados:

Método	@Transactional?	O que faz	Regra de negócio implementada
listar()	Não	Retorna todos os registros	—
buscarPorId(Long id)	Não	Busca um exercício por id, retorna Optional<Treinos	—
salvar(Treinos treino)	Sim	Persiste o objeto no banco	Verifica se o aluno associado existe antes de salvar
remover(Long id)	Sim	Remove o registro pelo id	Verifica se o exercício existe antes de atualizar
alterar(Long id, Treinos treino)	Sim	Atualiza campos do registro existente	Verifica existência antes de atualizar

Por que usar @Transactional?

No TreinosService, a anotação @Transactional é relevante nos métodos de escrita, pois operações como salvar() verificam a existência do aluno antes de persistir o treino. Sem essa anotação, uma falha no meio do processo poderia deixar o sistema com dados parcialmente persistidos, comprometendo a consistência dos programas de treino vinculados aos alunos.

7. CAMADA CONTROLLER

A camada Controller é a interface pública da aplicação. Ela expõe os endpoints HTTP da API REST, recebe requisições, delega ao Service e devolve a resposta com o código HTTP correto.

7.1 AlunoController

Anotações da classe:

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/alunos")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/alunos	listar()	200 OK	Retorna lista de todos os registros
GET	/alunos/{id}	buscarPorId(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado
POST	/alunos	salvar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/alunos/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/alunos/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /alunos/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto Aluno do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, como 200 OK, 201 Created ou 404 Not Found, em vez de retornar o objeto diretamente
ServletUriComponentsBuilder	Utilizado no método salvar() para construir o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do recurso recém-criado

7.2 AgendaTreinoController

Anotações da classe:

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/agenda")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/agenda	listar()	200 OK	Retorna lista de todos os registros
GET	/agenda/{id}	buscar(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado
POST	/agenda	criar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/agenda/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/agenda/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /agenda/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto AgendaTreino do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, retornando 201 Created ao criar um agendamento ou 404 Not Found quando o recurso não é localizado
ServletUriComponentsBuilder	Constrói o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do agendamento recém-criado

7.3 ExerciciosController

Anotações da classe:

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/exercicios")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/exercicios	listar()	200 OK	Retorna lista de todos os registros
GET	/exercicios/{id}	buscar(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado
POST	/exercicios	criar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/exercicios/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/exercicios/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /exercicios/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto Exercicios do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, garantindo que o cliente receba o status correto para cada operação
ServletUriComponentsBuilder	Constrói o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do exercício recém-cadastrado no catálogo

7.4 ExercicioTreinoController**Anotações da classe:**

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/exercicio-treino")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/exercicio-treino	listar()	200 OK	Retorna lista de todos os registros
GET	/exercicio-treino/{id}	buscar(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado

POST	/exercicio-treino	criar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/exercicio-treino/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/exercicio-treino/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /exercicio-treino/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto ExercicioTreino do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, garantindo que o cliente receba o status correto para cada operação
ServletUriComponentsBuilder	Constrói o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do exercício recém-cadastrado no catálogo

7.5 PlanosController

Anotações da classe:

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/planos")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/planos	listar()	200 OK	Retorna lista de todos os registros
GET	/planos/{id}	buscar(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado
POST	/planos	criar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/planos/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/planos/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /planos/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto Planos do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, garantindo que o cliente receba o status correto para cada operação
ServletUriComponentsBuilder	Constrói o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do exercício recém-cadastrado no catálogo

7.6 TreinosController

Anotações da classe:

Anotação	Finalidade
@RestController	Indica que a classe é um Controller REST; combina @Controller e @ResponseBody
@RequestMapping("/treinos")	Define o caminho base (prefixo) para todos os endpoints da classe

Endpoints implementados:

Método HTTP	Caminho	Método Java	Código HTTP	O que faz
GET	/treinos	listar()	200 OK	Retorna lista de todos os registros
GET	/treinos/{id}	buscar(id)	200 / 404	Retorna registro pelo id ou 404 se não encontrado
POST	/treinos	criar(body)	201 Created	Cria novo registro; retorna URI no cabeçalho Location
PATCH	/treinos/{id}	atualizar(id, body)	200 / 404	Atualiza registro existente
DELETE	/treinos/{id}	remover(id)	204 No Content	Remove registro; sem corpo na resposta

Explique o uso de cada elemento abaixo no seu Controller:

Elemento	Explicação
@PathVariable	Captura o id presente na URL e o passa como parâmetro ao método, por exemplo em /treinos/{id}
@RequestBody	Converte automaticamente o JSON enviado na requisição em um objeto Treinos do Java
ResponseEntity<T>	Permite definir o código HTTP da resposta de forma explícita, garantindo que o cliente receba o status correto para cada operação
ServletUriComponentsBuilder	Constrói o cabeçalho Location da resposta 201 Created, informando ao cliente a URI do exercício recém-cadastrado no catálogo

8. FLUXO COMPLETO DE UMA REQUISIÇÃO

Para ilustrar o fluxo completo de uma requisição no sistema AlphaCoach, foi escolhido o endpoint POST /alunos, responsável pelo cadastro de um novo aluno. Esse endpoint envolve todas as camadas da arquitetura e aplica uma regra de negócio relevante, a verificação de CPF duplicado, tornando-o um exemplo representativo do funcionamento do sistema.

Passo 1 — O cliente envia a requisição HTTP

O cliente envia uma requisição HTTP do tipo POST para o endereço /alunos, com os dados do novo aluno no corpo da requisição em formato JSON, contendo campos como nome, email, CPF, data de nascimento e plano.

Passo 2 — O Controller recebe a requisição

O AlunoController intercepta a requisição por meio da anotação @PostMapping. A anotação @RequestBody converte automaticamente o JSON recebido em um objeto do tipo Aluno. Em seguida, o Controller delega o processamento ao AlunoService, chamando o método salvar(aluno), sem aplicar nenhuma regra de negócio.

Passo 3 — O Service aplica as regras de negócio

O AlunoService recebe o objeto Aluno e inicia a validação. Antes de persistir o registro, ele invoca o método existsByCpf(cpf) do AlunoRepository para verificar se já existe um aluno cadastrado com o mesmo CPF. Caso o CPF já esteja em uso, o Service lança uma exceção e interrompe o fluxo, impedindo a duplicidade.

Passo 4 — O Service chama o Repository

Validadas as regras de negócio e confirmada a unicidade do CPF, o AlunoService chama o método save(aluno) do AlunoRepository, delegando a operação de persistência ao banco de dados.

Passo 5 — O Repository executa a operação no banco

O AlunoRepository, por meio do Hibernate e do JPA, traduz a chamada ao método save() em um comando SQL do tipo INSERT, que é executado diretamente na tabela alunos do PostgreSQL. O banco de dados retorna o registro persistido, com o id gerado automaticamente pela estratégia IDENTITY.

Passo 6 — O Repository retorna o resultado ao Service

O objeto Aluno já persistido, com seu id preenchido, é retornado ao AlunoService, que conclui a transação com sucesso.

Passo 7 — O Service retorna o resultado ao Controller

O AlunoService repassa o objeto persistido ao AlunoController, encerrando sua responsabilidade no fluxo.

Passo 8 — O Controller monta e retorna a resposta HTTP

O AlunoController utiliza o ServletUriComponentsBuilder para construir a URI do recurso recém-criado e a inclui no cabeçalho Location da resposta. A resposta é retornada ao cliente com o código HTTP 201 Created, indicando que o recurso foi criado com sucesso e informando onde ele pode ser acessado.

9. TESTES REALIZADOS

A API do sistema AlphaCoach foi testada utilizando o Postman, ferramenta amplamente utilizada para validação de APIs REST. Foram realizados testes em todos os endpoints disponíveis, verificando tanto os cenários de sucesso quanto os cenários de erro, como entradas inválidas e recursos inexistentes.

9.1 AlunoController — /alunos

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /alunos	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /alunos	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /alunos /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /alunos /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /alunos /{id}	Remoção	id válido	204 No Content	Passou
PATCH /alunos/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

9.2 TreinosController — /treinos

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /treinos	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /treinos	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /treinos /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /treinos /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /treinos /{id}	Remoção	id válido	204 No Content	Passou
PATCH /treinos/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

9.3 ExerciciosController — /exercicios

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /exercicios	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /exercicios	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /exercicios /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /exercicios /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /exercicios /{id}	Remoção	id válido	204 No Content	Passou
PATCH /exercicios/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

9.4 ExercicioTreinoController — /exercicio-treino

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /exercicio-treino	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /exercicio-treino	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /exercicio-treino /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /exercicio-treino /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /exercicio-treino /{id}	Remoção	id válido	204 No Content	Passou
PATCH /exercicio-treino/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

9.5 PlanosController — /planos

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /planos	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /planos	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /planos /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /planos /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /planos /{id}	Remoção	id válido	204 No Content	Passou
PATCH /planos/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

9.6 AgendaTreinoController — /agenda

Endpoint	Tipo de Teste	Entrada (Request)	Saída esperada	Resultado
POST /agenda	Criação com sucesso	JSON com dados válidos	201 Created + URI	Passou
POST /agenda	Criação com dados inválidos	JSON com campo obrigatório vazio	400 Bad request	Passou
GET /agenda /{id}	Busca existente	id = 1	200 + objeto JSON	Passou
GET /agenda /{id}	Busca inexistente	id = 9999	404 Not Found	Passou
DELETE /agenda /{id}	Remoção	id válido	204 No Content	Passou
PATCH /agenda/{id}	Atualização	JSON com campos a alterar	200 OK + objeto atualizado	Passou

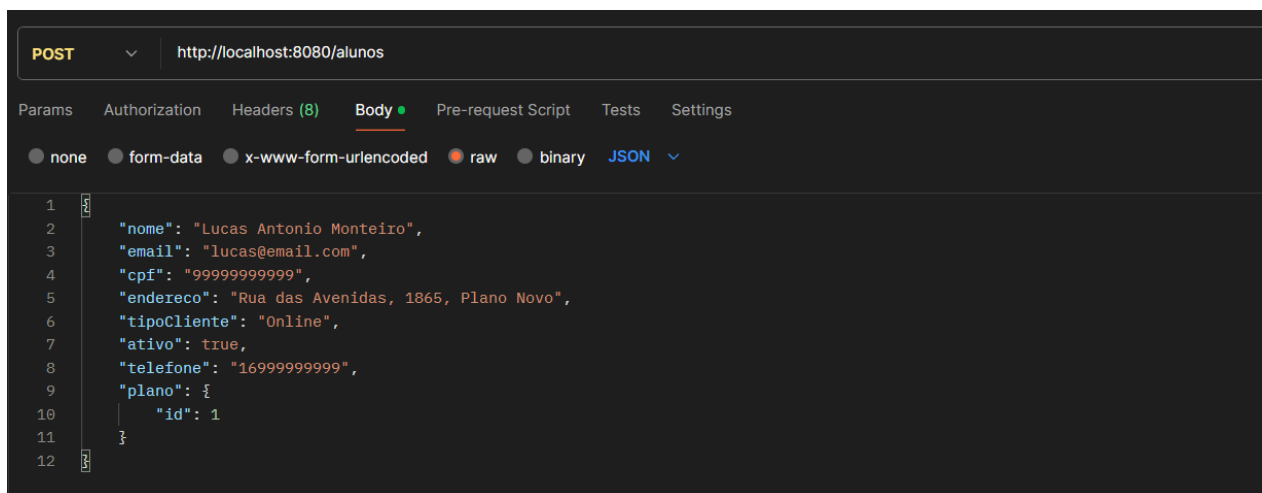
Ferramenta utilizada para testes:

Os testes foram realizados utilizando o Postman. Cada endpoint foi testado manualmente, verificando o código HTTP retornado, o corpo da resposta em JSON e o cabeçalho Location nas respostas de criação. Os dados utilizados nas

requisições foram baseados nos registros reais inseridos no banco de dados durante o desenvolvimento, conforme evidenciado pelas tabelas do PostgreSQL apresentadas a seguir.

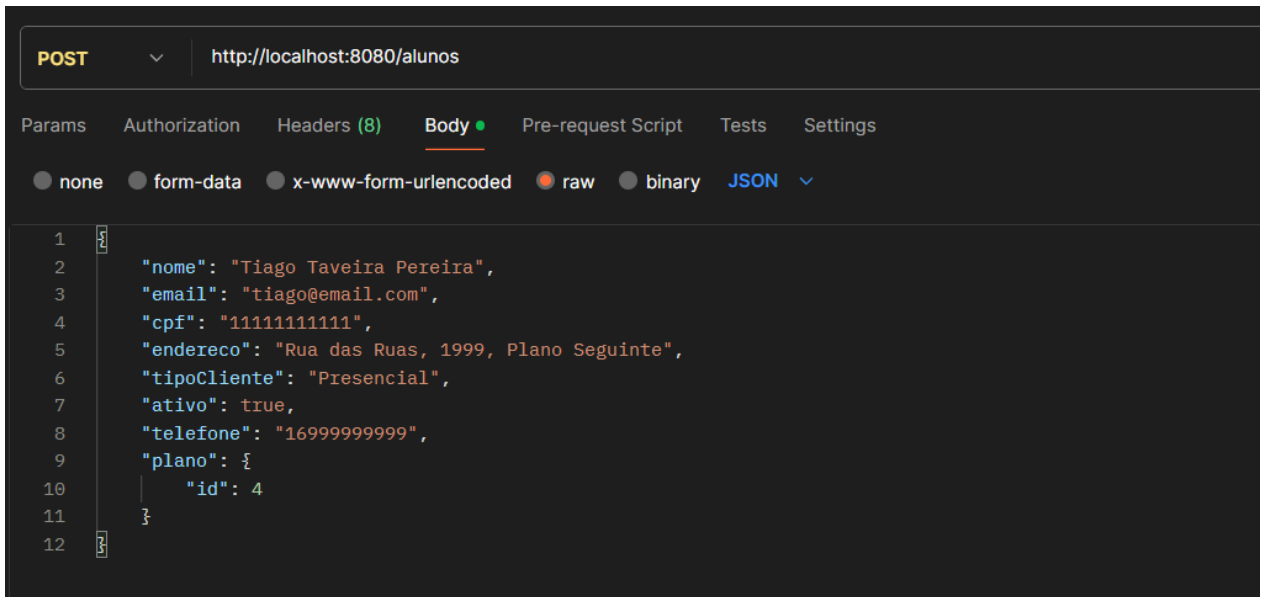
As imagens abaixo ilustram os dados persistidos no banco de dados após a execução dos testes, confirmando que as operações de criação foram realizadas com sucesso e que os relacionamentos entre as entidades foram corretamente estabelecidos.

Figura 1 - Print do postman



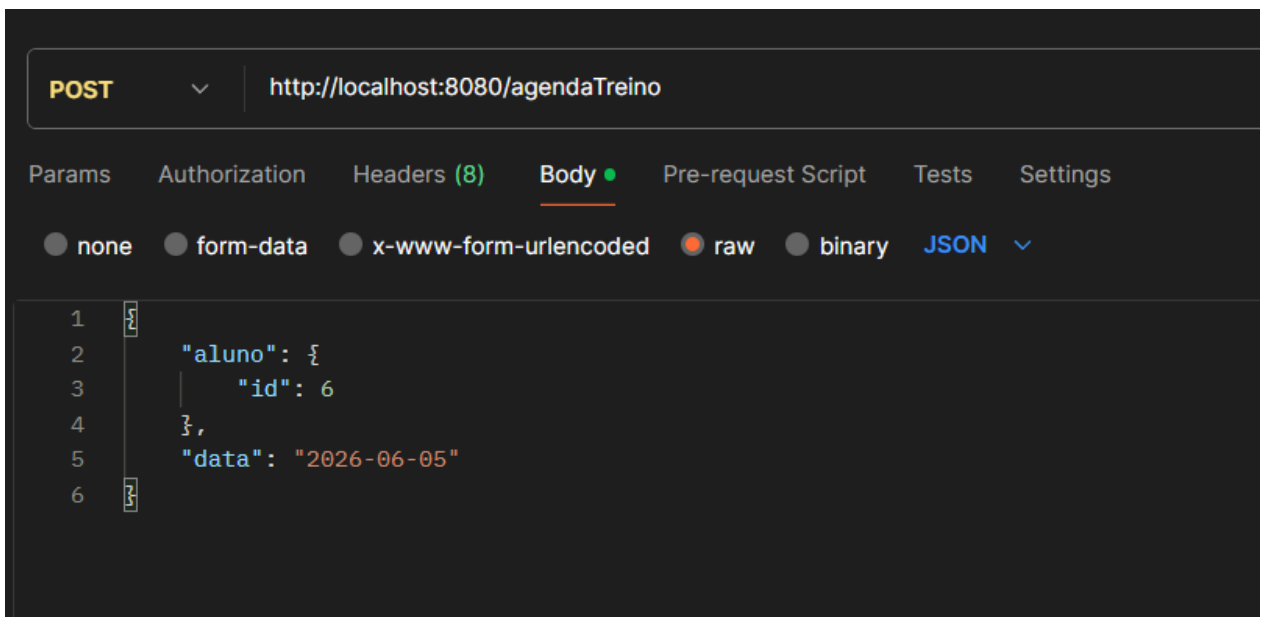
Fonte: Elaborado pelos autores (2026)

Figura 2 - Print do postman



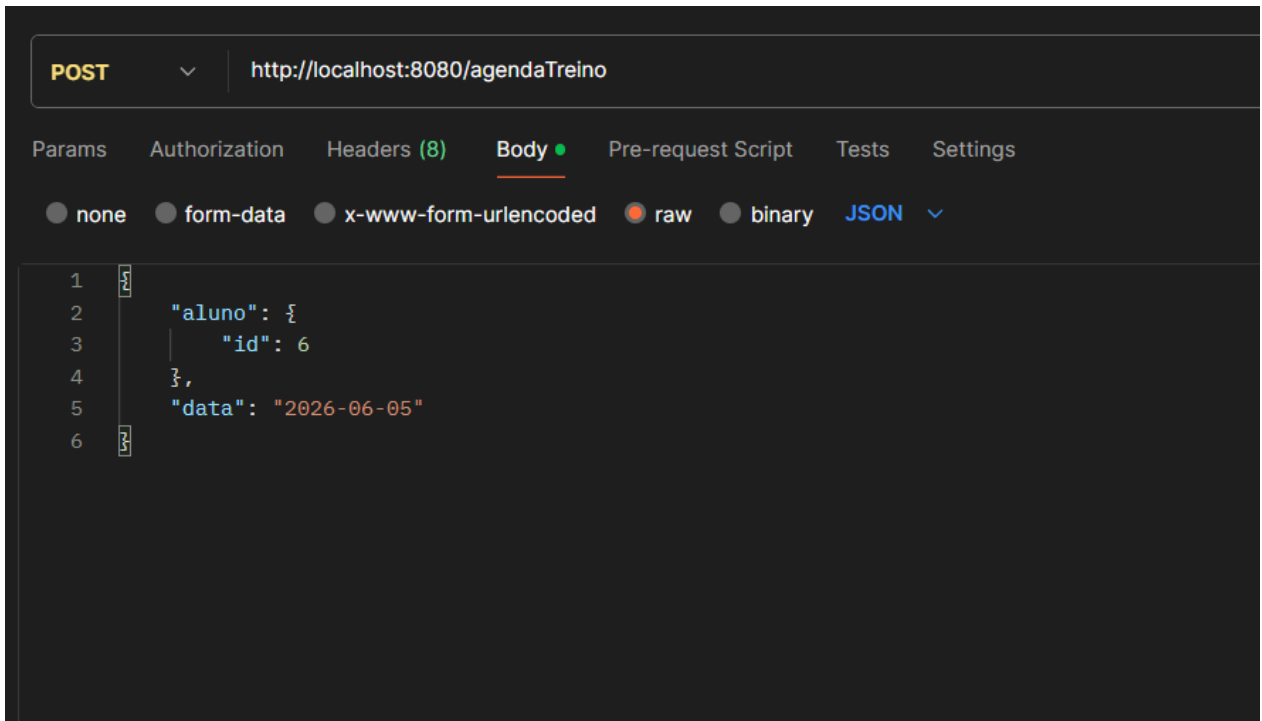
Fonte: Elaborado pelos autores (2026)

Figura 3 - Print do postman



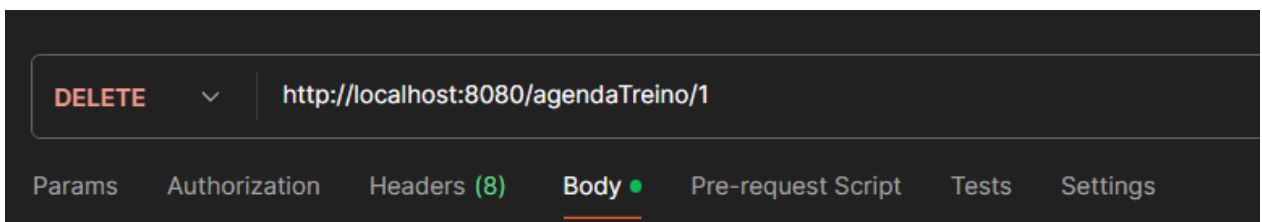
Fonte: Elaborado pelos autores (2026)

Figura 4 - Print do postman



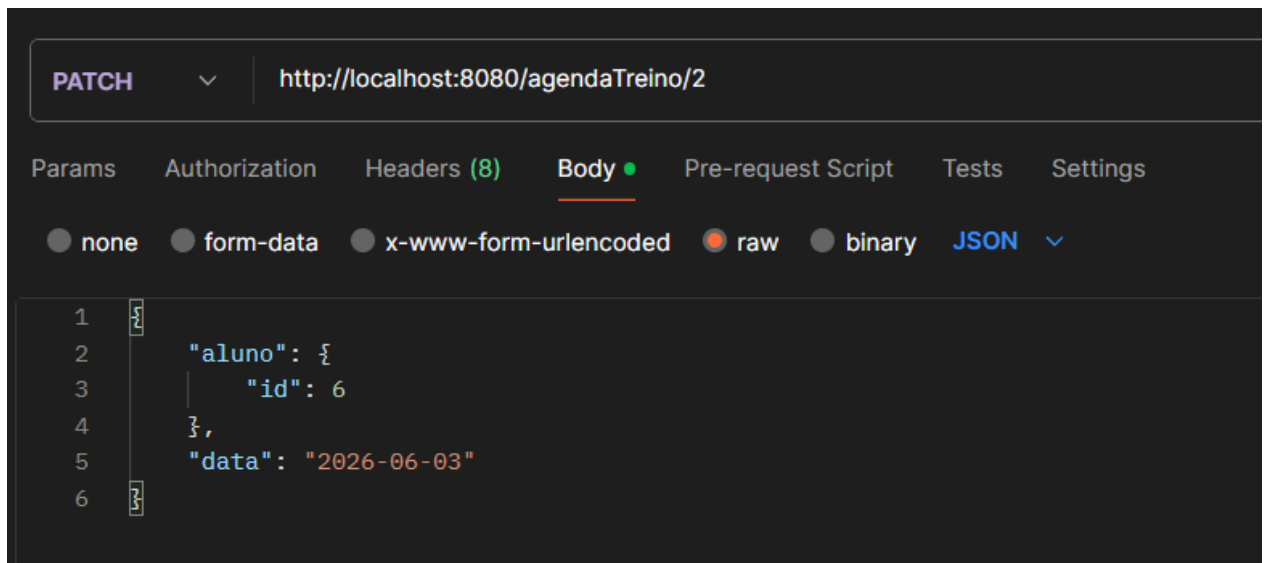
Fonte: Elaborado pelos autores (2026)

Figura 5 - Print do postman



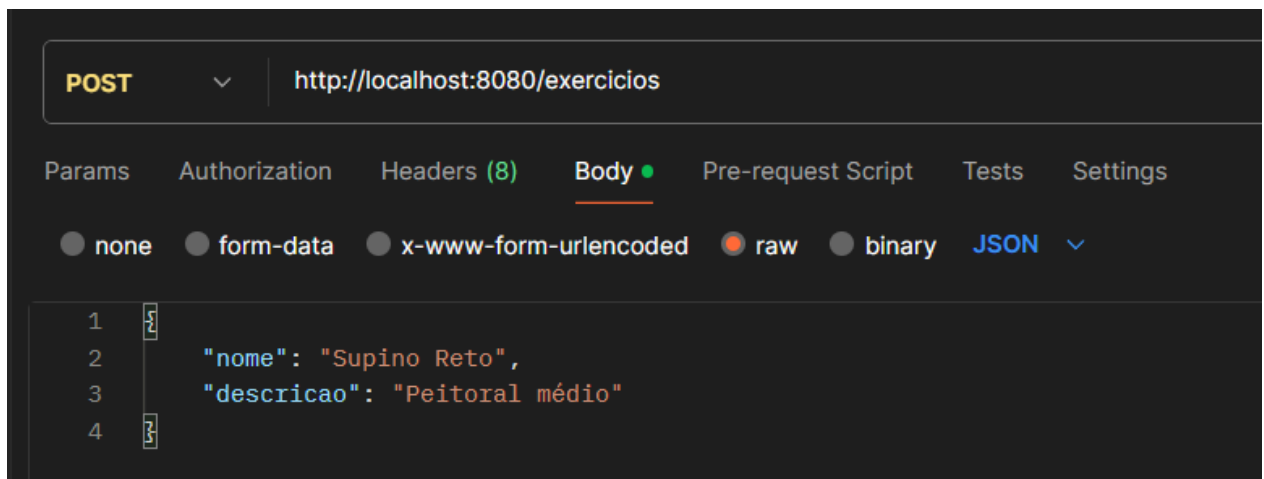
Fonte: Elaborado pelos autores (2026)

Figura 6 - Print do postman



Fonte: Elaborado pelos autores (2026)

Figura 7 - Print do postman



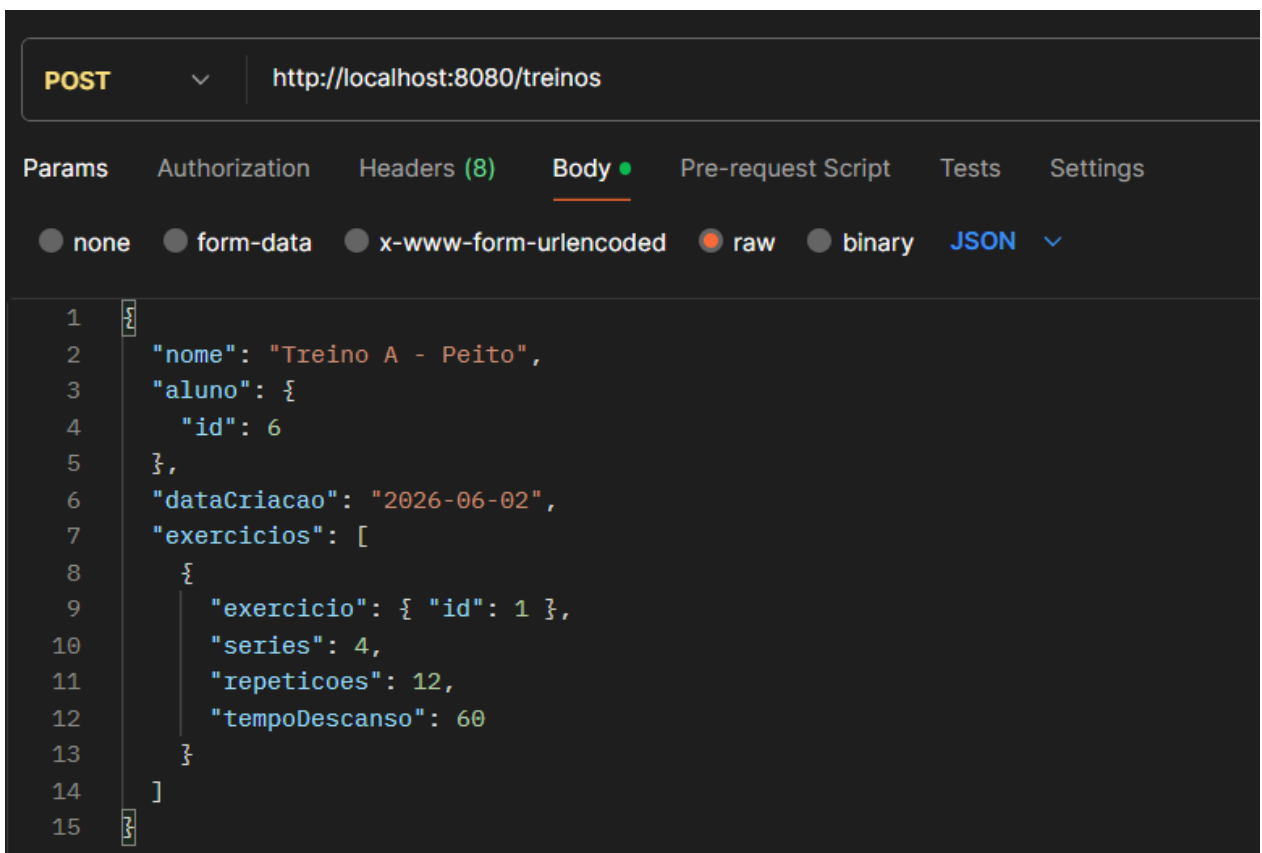
Fonte: Elaborado pelos autores (2026)

Figura 8 - Print do postman



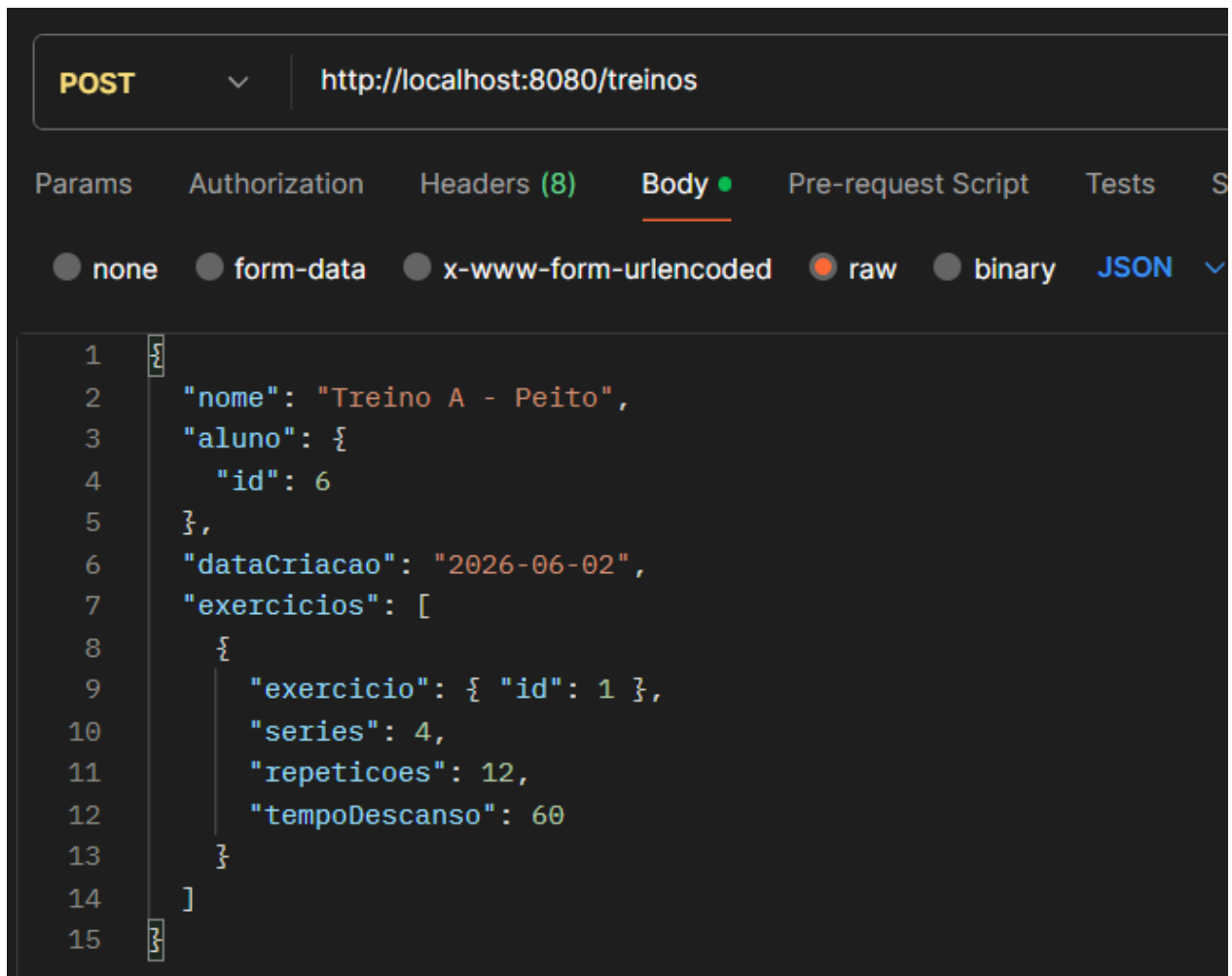
Fonte: Elaborado pelos autores (2026)

Figura 9 - Print do postman



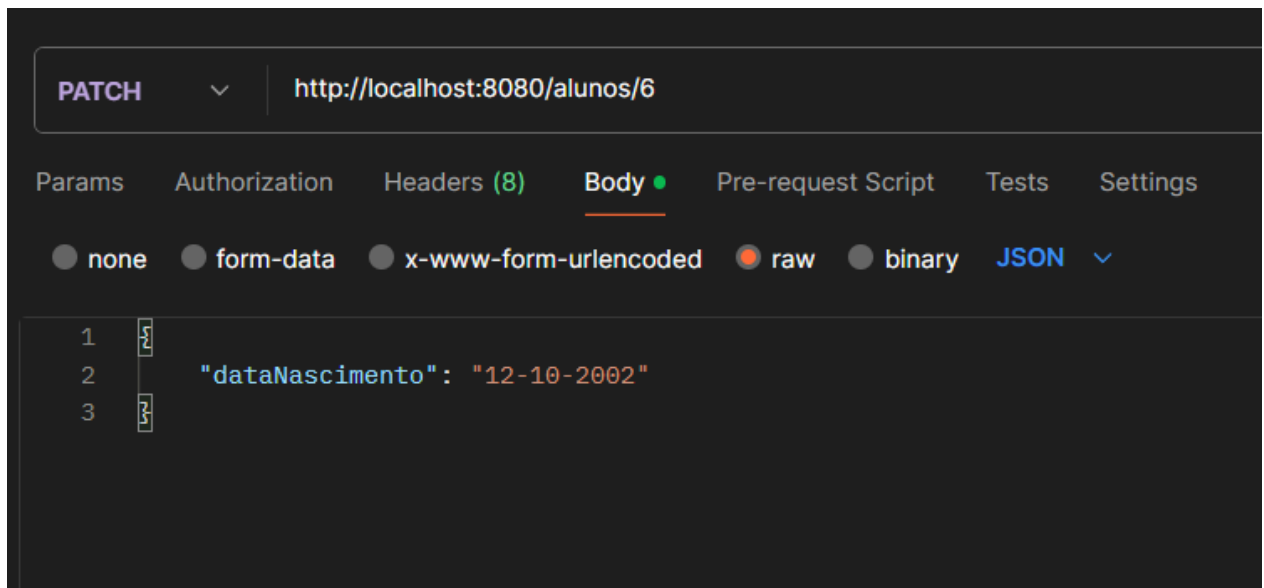
Fonte: Elaborado pelos autores (2026)

Figura 10 - Print do postman



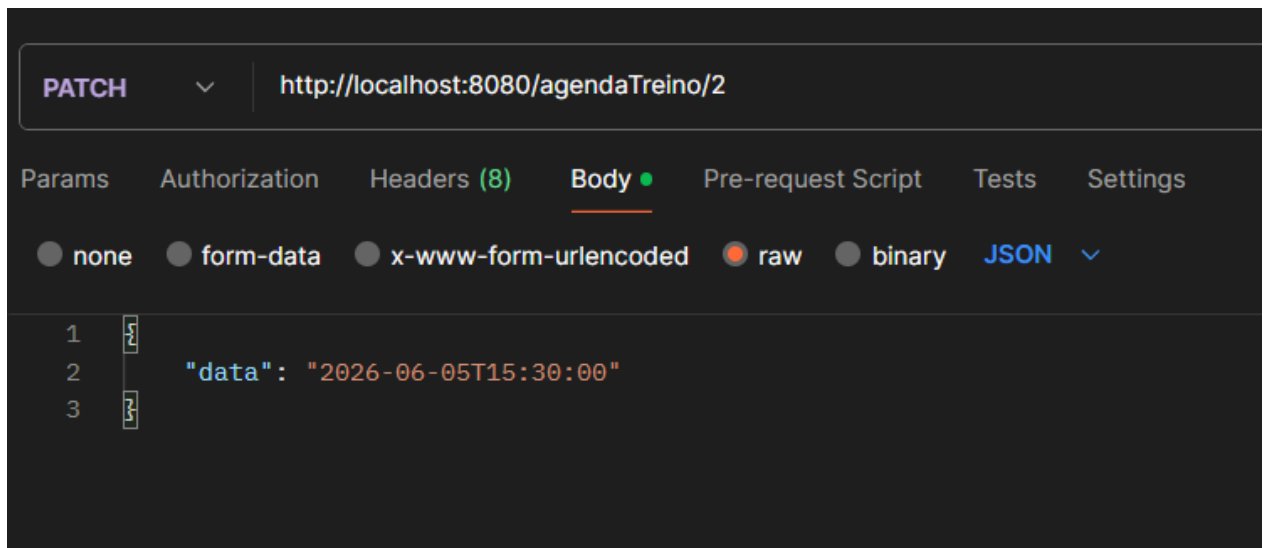
Fonte: Elaborado pelos autores (2026)

Figura 11 - Print do postman



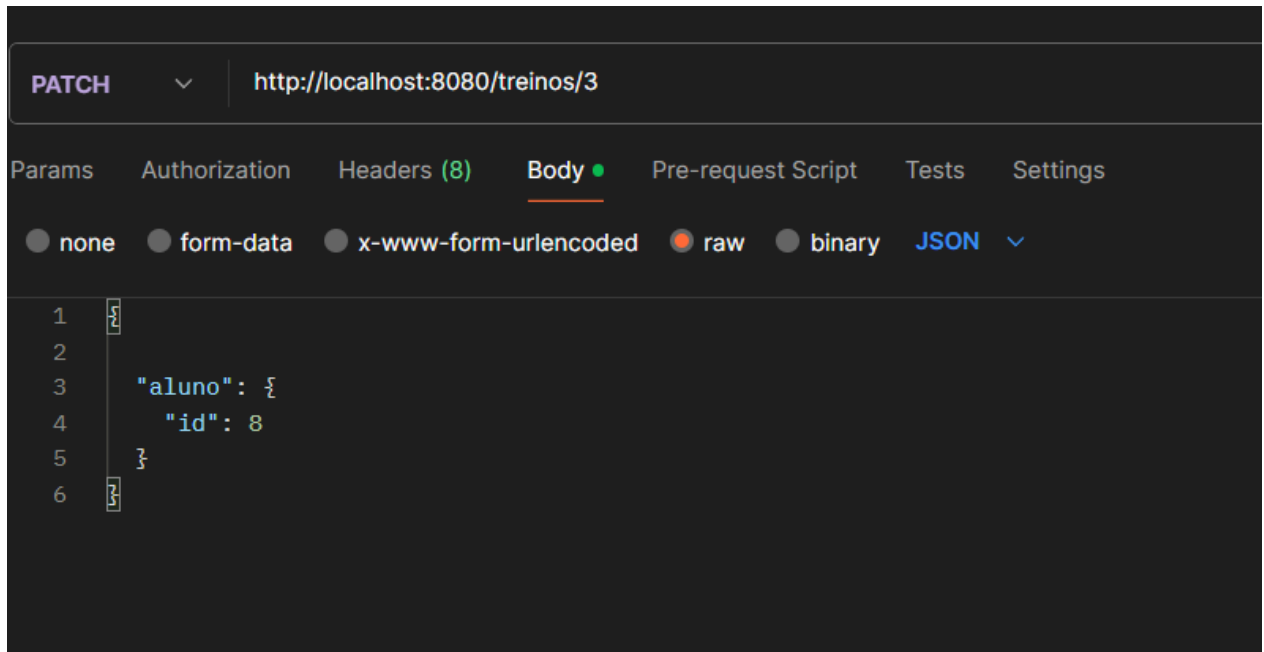
Fonte: Elaborado pelos autores (2026)

Figura 12 - Print do postman



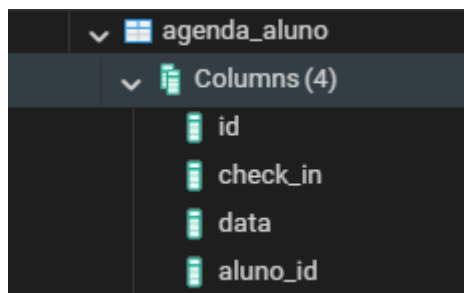
Fonte: Elaborado pelos autores (2026)

Figura 13 - Print do postman



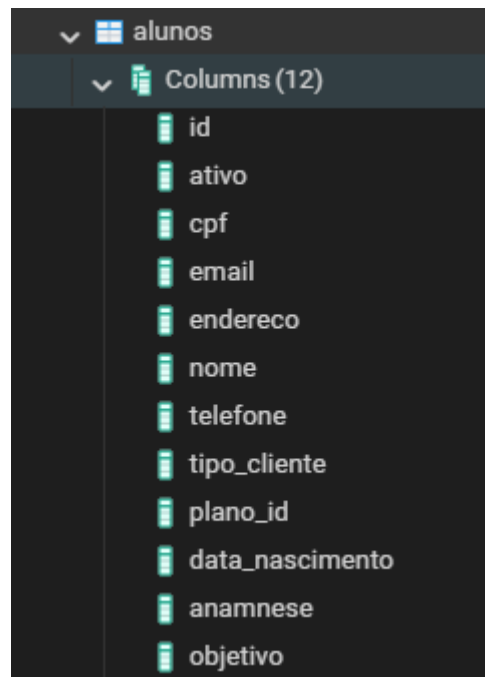
Fonte: Elaborado pelos autores (2026)

Figura 14 - Print do postgresql



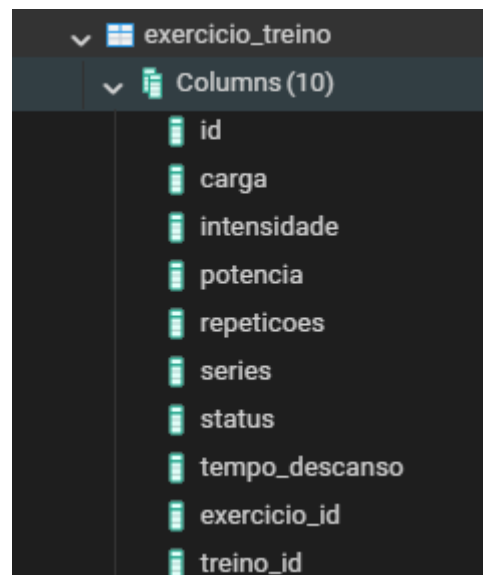
Fonte: Elaborado pelos autores (2026)

Figura 15 - Print do postgresql



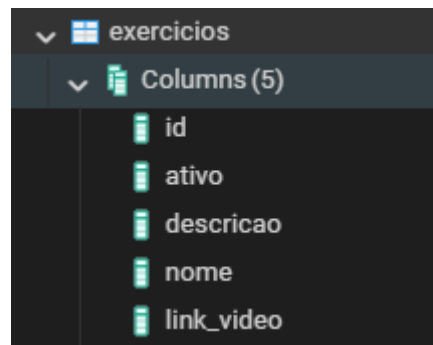
Fonte: Elaborado pelos autores (2026)

Figura 16 - Print do postgresql



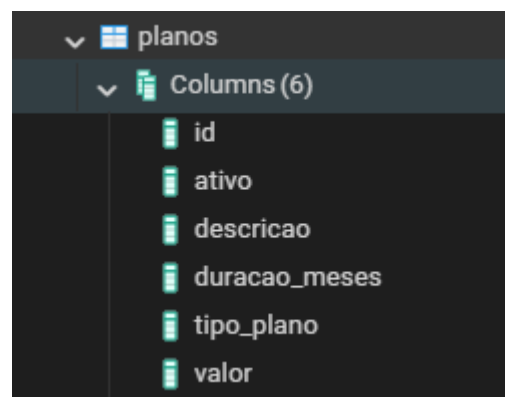
Fonte: Elaborado pelos autores (2026)

Figura 17 - Print do postgresql



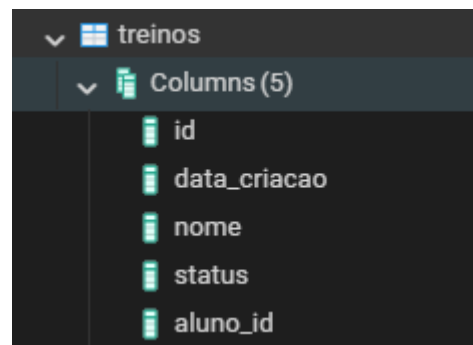
Fonte: Elaborado pelos autores (2026)

Figura 18 - Print do postgresql



Fonte: Elaborado pelos autores (2026)

Figura 19 - Print do postgresql



Fonte: Elaborado pelos autores (2026)

10. CONCLUSÃO

O desenvolvimento do sistema AlphaCoach representou uma experiência prática fundamental para a consolidação dos conhecimentos adquiridos ao longo do curso. Trabalhar com um domínio real, a gestão de alunos e treinos do Personal Trainer Tiago, tornou o aprendizado mais significativo, pois cada decisão técnica tinha um impacto direto em um problema concreto que o profissional enfrenta no dia a dia.

Entre os principais aprendizados que este projeto proporcionou, destaca-se a compreensão aprofundada do mapeamento objeto-relacional com JPA e Hibernate. Antes do projeto, o entendimento sobre como uma classe Java se transforma em uma tabela no banco de dados era predominantemente teórico. A prática de configurar anotações como `@OneToMany`, `@ManyToOne` e `@JoinColumn`, observar os comandos SQL gerados automaticamente pelo Hibernate e entender o comportamento do `CascadeType` foram aprendizados que só a experiência prática é capaz de consolidar. Da mesma forma, compreender a diferença entre ser o lado proprietário ou o lado inverso de um relacionamento, e o impacto disso na geração das chaves estrangeiras, foi um conhecimento que ficou muito mais claro ao lidar com entidades reais como `ExercicioTreino` e `AgendaTreino`.

As maiores dificuldades encontradas durante o desenvolvimento estavam relacionadas justamente aos relacionamentos entre entidades. O problema de referências circulares durante a serialização JSON, em que `Aluno` referenciava `AgendaTreino`, que por sua vez referenciava `Aluno` novamente, gerando um loop infinito na resposta da API, foi um dos obstáculos mais desafiadores. A solução foi aplicar a anotação `@JsonIgnoreProperties` nas entidades envolvidas, interrompendo o ciclo de serialização. Outro ponto de dificuldade foi a configuração correta do banco de dados PostgreSQL integrado ao Spring Boot, especialmente no que diz respeito à estratégia de geração de tabelas e ao dialeto do Hibernate, que exigiu atenção redobrada no arquivo `application.properties`.

A adoção da arquitetura em camadas impactou positivamente a organização e a qualidade do código de forma perceptível ao longo de todo o desenvolvimento. À medida que o projeto crescia, com novas entidades, novos endpoints e novas regras de negócio, a separação entre `Controller`, `Service` e `Repository` fez com que cada adição fosse feita de forma previsível e segura, sem o risco de comprometer funcionalidades já implementadas. A legibilidade do código também se destacou: ao abrir qualquer classe do projeto, era imediatamente claro qual era sua responsabilidade, o que facilitou tanto a depuração de erros quanto a evolução das funcionalidades.

Quanto à camada mais importante, considera-se a camada `Service` como a mais relevante do projeto. É nela que reside a inteligência do sistema, as regras que definem o comportamento do negócio, como a verificação de CPF duplicado no

cadastro de alunos ou a validação de conflito de horários no agendamento de treinos. O Controller pode ser substituído por outro tipo de interface, o Repository pode ter sua implementação trocada, mas as regras de negócio contidas no Service são o coração da aplicação e o que de fato agrega valor ao Personal Trainer Tiago no uso do sistema.

Se o projeto fosse refeito, algumas decisões seriam tomadas de forma diferente. A principal delas seria a adoção de DTOs (Data Transfer Objects) desde o início, separando os objetos de entrada e saída da API das entidades do banco de dados. Essa prática evitaria os problemas de referência circular de forma mais elegante e daria maior controle sobre quais campos são expostos em cada endpoint. Além disso, seria implementado um tratamento de exceções centralizado com `@ControllerAdvice`, padronizando as respostas de erro da API em vez de deixá-las espalhadas em cada Service. Por fim, a escrita de testes automatizados com JUnit e Mockito desde o início do desenvolvimento, e não apenas testes manuais via Postman, garantiria maior segurança a cada nova funcionalidade adicionada ao sistema.

De forma geral, o projeto AlphaCoach cumpriu seu papel como experiência de aprendizado completa, integrando conceitos de modelagem de domínio, persistência de dados, arquitetura de software e desenvolvimento de APIs REST em um sistema coeso e funcional, capaz de resolver um problema real de um profissional autônomo da área de educação física.

REFERÊNCIAS

PARASCHIV, Eugen. *Introduction to Spring Data JPA*. Disponível em: <https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa>. Acesso em: 10 maio 2026.

PARASCHIV, Eugen. *REST API with Spring and Java Config*. Disponível em: <https://www.baeldung.com/building-a-restful-web-service-with-spring-and-java-based-configuration>. Acesso em: 15 maio 2026.

PARASCHIV, Eugen. *Spring Boot Transaction Management*. Disponível em: <https://www.baeldung.com/transaction-configuration-with-jpa-and-spring>. Acesso em: 18 maio 2026.

HIBERNATE. *Hibernate ORM Documentation*. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 02 junho 2026.

POSTGRESQL. *PostgreSQL 16 Documentation*. Disponível em: <https://www.postgresql.org/docs/current/>. Acesso em: 05 junho 2026.

SPRING. *Spring Boot Reference Documentation*. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: 05 junho 2026.

SPRING. *Spring Data JPA – Reference Documentation*. Disponível em: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>. Acesso em: 13 junho 2026.

SPRING. *Spring Framework – Web MVC*. Disponível em: <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html>. Acesso em: 13 junho 2026.